

Sticky Note Research

1. How HTTP applications preserve the state of an application across multiple request-response cycles, especially concerning user authentication and session management

HTTP is a stateless protocol, which means the server treats every request as a new one and does not automatically remember previous requests from the same user. If websites didn't have a way to keep track of users, people would have to log in again every time they opened a new page. To solve this problem, web applications use cookies, sessions, and authentication tokens to remember information between requests.

In Django, this is mainly handled through the session framework. When a user logs in, Django creates a unique session ID and stores it in a cookie in the user's browser. Each time the browser sends another request, it automatically includes this cookie. Django then uses the session ID to find the user's session data stored on the server. This allows the application to recognize the user without asking them to log in again.

The session can store information such as the user's identity, items in a shopping cart, or personal preferences. Django also includes security features such as CSRF protection, signed cookies, session expiry settings, and safeguards that help reduce

the risk of session hijacking. These features make it possible to keep users logged in while protecting their data.

2. Investigate and document the procedures for performing Django database migrations to a server-based relational database like MariaDB.

Django migrations are used to keep the database structure in sync with the models in a project. When using a server-based database such as MariaDB, the first step is to install a compatible database driver, for example `mysqlclient`, and configure the database connection in the project's `settings.py` file. This includes providing the database name, username, password, host, port, and the correct database engine.

Before running any migrations, the MariaDB database must already exist, and the database user must have permission to create and modify tables. After making changes to the models, the `python manage.py makemigrations` command creates migration files that describe those changes. Running `python manage.py migrate` then applies the changes to the MariaDB database by creating or updating the necessary tables.

Django keeps track of which migrations have already been applied by storing this information in the `django_migrations` table. This means migrations are only applied once and in the correct order. Using migrations makes it much easier to manage database changes across development, testing, and production

environments, while helping ensure that everyone working on the project has the same database structure.